

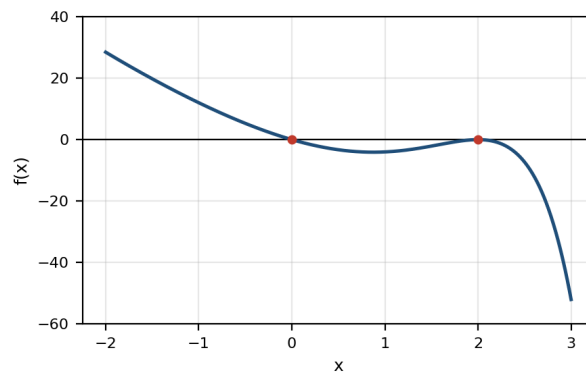
Exercise #4 – Solution Proposal

TMA4130/TMA4135 — Autumn 2026

Problem 1 — Bisection Method

Writing $t = 3^x$, $f(x)$ collects into $f(x) = 2(x-2)^2(1-3^x)$. So f has a **simple root at $x = 0$** and a **double root at $x = 2$** , and no others.

a) Plot on $[-2, 3]$



b) Four bisection steps on $[-2, 3]$

$f(-2) > 0$, $f(3) < 0$, so this brackets the root $x = 0$:

n	interval	c	sign $f(c)$	new interval
1	$[-2, 3]$	0.5	–	$[-2, 0.5]$
2	$[-2, 0.5]$	–0.75	+	$[-0.75, 0.5]$
3	$[-0.75, 0.5]$	–0.125	+	$[-0.125, 0.5]$
4	$[-0.125, 0.5]$	0.1875	–	$[-0.125, 0.1875]$

After 4 steps: $[a_4, b_4] = [-0.125, 0.1875]$, so $\tilde{x} = (a_4 + b_4)/2 = \mathbf{0.03125}$, error bound $= (b_4 - a_4)/2 = \mathbf{0.15625}$ — this locates the root $x = 0$.

c) Iterations for error $< 10^{-3}$

Error bound after n steps is $(b_0 - a_0)/2^{n+1} = 5/2^{n+1}$. Requiring this $< 10^{-3}$ gives $2^{n+1} > 5000$, i.e. $n+1 > \log_2(5000) \approx 12.29$, so **$n = 12$** iterations suffice.

d) The root at $x = 2$

$f(2) = 0$, but with even multiplicity (from the squared factor), so f does not change sign there (e.g. $f(1) = -4$, $f(2.5) \approx -7.3$, both negative). Bisection needs $f(a) \cdot f(b) < 0$ to bracket a root, so it can never isolate $x = 2$.

e) Python

```
def f(x): return 2*(x-2)**2*(1-3**x)

def bisection(f, a, b, tol=1e-4):
    fa, n = f(a), 0
    while (b-a)/2 > tol:
        c, fc = (a+b)/2, f((a+b)/2)
        if fa*fc < 0: b = c
        else: a, fa = c, fc
        n += 1
    return (a+b)/2, n

print(bisection(f, -2, 3, 1e-4)) # => (4.58e-05, 15)
```

Root $\approx 4.6 \times 10^{-5} \approx 0$ after **15 iterations** — matches part c) for $\text{tol} = 10^{-4}$.

Problem 2 — Fixed-Point Method

$f(x) = x^3 - 3x + 2 = (x-1)^2(x+2)$: double root $x = 1$, simple root $x = -2$.

a) Derivatives

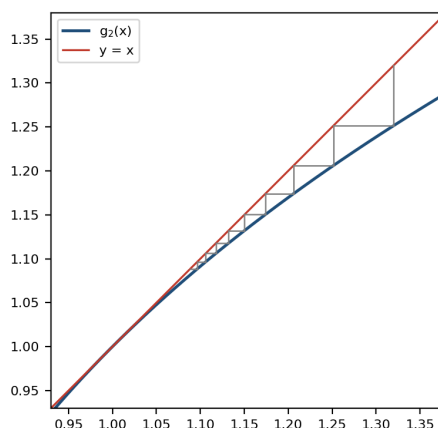
$$g_1'(x) = x^2, \quad g_2'(x) = \frac{1}{|3x-2|^{2/3}}, \quad g_3'(x) = \frac{-4(x^2-x-1)}{(x^2+1)^2}$$

	g_1'	g_2'	g_3'
at $x=1$	1	1	1
at $x=-2$	4	0.25	-0.80

All three satisfy $g_1'(1) = 1$ exactly, since $x=1$ is a double root ($f'(1)=0$) — none is a genuine contraction there. g_1 is only safe on $(-1,1)$, so it is unusable far from either root. Between g_2 and g_3 , g_3 has the smaller slope at both roots.

b) Iterating from $x^{(0)}=4$ until $|x^{(k+1)} - x^{(k)}| < 10^{-6}$

g	outcome	iterations
$g_1(x) = (x^3+2)/3$	diverges	—
$g_2(x) = (3x-2)^{1/3}$	$\rightarrow x \approx 1$	1005
$g_3(x) = (4x-2)/(x^2+1)$	$\rightarrow x \approx -2$	61



Cobweb of g_2 near $x=1$: the curve is tangent to $y=x$, so steps crawl instead of collapsing — this is why it takes 1005 iterations.

Comment. g_1 diverges ($x^{(0)}=4$ is outside $(-1,1)$). g_2 does reach $x=1$, but very slowly: multiplicity 2 forces $g_2' \rightarrow 1$ there, killing the usual geometric rate. g_3 converges fast — but overshoots past 1 and lands on the *other* root, $x = -2$, where $|g_3'| = 0.8$ gives comfortable convergence. Lesson: $|g'| < 1$ only guarantees local convergence, and a double root can make it very slow even when satisfied.

Problem 3 — Newton's Method

$f(x) = x - e^{-x}$, $f'(x) = 1 + e^{-x}$.

$$x_{n+1} = x_n - \frac{x_n - e^{-x_n}}{1 + e^{-x_n}}$$

a) Two steps from $x_0 = 2$

$$x_1 = 2 - (2 - e^{-2})/(1 + e^{-2}) \approx \mathbf{0.3576}$$

$$x_2 = x_1 - (x_1 - e^{-x_1})/(1 + e^{-x_1}) \approx \mathbf{0.5587}$$

b) Python

```
def fN(x): return x - np.exp(-x)
def fNp(x): return 1 + np.exp(-x)

def newton(x0, tol=1e-6):
    x, n = x0, 0
    while abs(fN(x)) > tol:
```

```

        x = x - fN(x)/fNp(x); n += 1
    return x, n

```

```

print('rooter(3, 0)) = %s' % (0.5671432904, 5)

```

Converges to $x \approx \mathbf{0.5671433}$ (the Omega constant) in **5 iterations**.

Problem 4

$$e^{e^{-x}} = \frac{5x}{3}$$

a) Uniqueness

e^{-x} is strictly decreasing, so $e^{e^{-x}}$ is strictly decreasing, while $5x/3$ is strictly increasing — their difference is strictly decreasing, so **at most one** root. As $x \rightarrow -\infty$ it $\rightarrow +\infty$, as $x \rightarrow +\infty$ it $\rightarrow -\infty$, so by IVT there is **exactly one** root x^* .

b) Error bound

The code iterates $x = g(x)$ with $g(x) = (3/5)e^{e^{-x}}$, $g'(x) = -e^{-x}g(x)$. It converges after **16 steps** to $x \approx 0.9007279$ (last step size $\approx 9.33 \times 10^{-7}$). On $[0.8, 1.0]$, $L = \max|g'| \approx 0.4225 < 1$, so the fixed-point bound gives

$$e = |x^* - x_{16}| \leq \frac{L}{1-L} |x_{16} - x_{15}| \approx 6.8 \times 10^{-7}$$

i.e. $x^* \approx \mathbf{0.900728}$, accurate to about 6–7 significant digits.

All numerical results verified with Python/NumPy.